



1. Агент как цикл, а не как одно действие

Когда мы говорим о логике работы агента, важно перестать воспринимать его как систему, которая получает один запрос и сразу выдаёт окончательный результат. Для агента характерен циклический процесс. Он получает цель, определяет следующее действие, использует инструмент, анализирует полученный результат и уже на основании новой информации решает, что делать дальше.

Например, при задаче «проверь проект после обновления зависимости» агент сначала может изучить состояние репозитория. После этого он увидит изменённые файлы, затем прочитает наиболее важные из них, запустит тесты и только после результатов тестирования сможет сделать вывод о корректности изменений. То есть каждое следующее действие зависит от результата предыдущего.

В Hermes этот принцип особенно хорошо заметен при работе с инструментами. Модель формирует вызов инструмента, Hermes выполняет его и возвращает результат обратно модели. После этого начинается следующая итерация. Именно поэтому агент способен не просто заранее описать последовательность действий, а корректировать её во время выполнения задачи. (github.com)

2. Планирование, исполнение и проверка

Удобно мысленно разделять агентный цикл на три логические стадии: планирование, исполнение и проверку. Это не обязательно три отдельных агента, как в нашей предыдущей работе с Planner, Coder и Reviewer. Один Hermes Agent тоже может несколько раз проходить через эти стадии внутри одной задачи.

На этапе планирования агент определяет, какой информации ему не хватает и какое действие имеет смысл выполнить следующим. Во время исполнения он вызывает конкретный инструмент. После этого появляется проверка: агент сопоставляет полученный результат со своей целью и определяет, достигнута ли она.

Если после проверки становится понятно, что результата недостаточно, цикл повторяется. Например, тест завершился ошибкой. Это не обязательно означает завершение задачи. Ошибка становится новой информацией, которую агент может исследовать.

Именно в этой точке агент отличается от заранее написанного жёсткого сценария. Скрипт обычно выполняет заранее определённую последовательность. Агент может менять дальнейшие действия в зависимости от результатов предыдущих шагов.

3. Ошибка инструмента как часть рабочего контекста

При обычном ручном использовании терминала ошибка команды отображается пользователю, после чего человек самостоятельно решает, что делать дальше. В агентной системе ошибку должен получить сам агент.

Hermes оборачивает выполнение инструментов так, чтобы сбой обработчика не приводил к неконтролируемому разрушению всего агентного цикла. Ошибка преобразуется в структурированный результат и возвращается модели. Таким образом, модель видит не просто отсутствие ответа, а сообщение о том, что конкретное действие завершилось неудачно. (github.com)

Для нас это важный принцип. Ошибка инструмента и ошибка всей задачи — не одно и то же.

Представим, что агент пытается прочитать файл, которого больше не существует. Первый вызов завершается ошибкой. Агент может проверить структуру каталогов, обнаружить новое имя файла и продолжить работу.

Но если ошибка означает, что требуемый ресурс принципиально недоступен, бесконечно повторять одно и то же действие нет смысла. Хорошая логика агента должна различать исправимую ошибку, временный сбой и условие, при котором работу необходимо прекратить.

4. Таймауты и зависшие действия

Не каждая проблема возвращает явную ошибку. Иногда команда просто работает слишком долго или процесс зависает. Поэтому агентная система должна иметь ограничения по времени выполнения.

В Hermes для терминальных команд можно настроить timeout. В конфигурации terminal по умолчанию используется ограничение времени выполнения команды, а долгоживущие процессы можно запускать в background-режиме и затем контролировать отдельно через инструмент `process`. Это важно, потому что сервер, watcher или длительная сборка не должны блокировать весь агентный цикл как обычная короткая команда. (github.com)

Отдельные механизмы имеют собственные лимиты. Например, `execute_code` по умолчанию ограничивает выполнение скрипта пятью минутами, после чего процесс завершается. Для субагентов действует другой подход: у них есть лимит количества итераций, а жёсткого общего wall-clock timeout по умолчанию сейчас нет, чтобы долгие, но реально работающие исследования не завершались преждевременно. (github.com) (github.com)

Получается, что таймаут — это не просто защита от медленной программы. Это граница, которая не позволяет одному неудачному действию бесконечно удерживать весь процесс.

5. Отказоустойчивость агентного процесса

Отказоустойчивость означает, что отдельный неудачный шаг не должен автоматически разрушать всю работу, если существует безопасный способ продолжения.

Например, один субагент может завершиться ошибкой из-за недоступного провайдера или неправильной конфигурации модели. Hermes возвращает родительскому агенту статус `failed` и описание причины. Родитель после этого может решить, можно ли повторить действие, выбрать другой путь или сообщить пользователю о невозможности завершить конкретную часть задачи. Ошибка дочернего агента не должна исчезать незаметно. (github.com)

Отказоустойчивость при этом не означает автоматическое повторение любой операции. Представьте команду, которая меняет данные или отправляет запрос во внешнюю систему. Если агент не уверен, выполнилось действие или нет, бездумный повтор может привести к двойной операции.

Поэтому устойчивый агент должен учитывать тип действия. Чтение файла обычно безопасно повторить. Создание платежа, удаление ресурса или изменение production-системы требуют гораздо более осторожной логики.

6. Что такое Tool в Hermes

Теперь перейдём к теме Skills и Tools. Tool — это конкретная функция, которую языковая модель может попросить Hermes выполнить.

Сама модель не выполняет команду терминала и не читает файл напрямую. Она формирует структурированный вызов инструмента. Hermes принимает его, проверяет параметры, запускает соответствующий обработчик и возвращает результат модели.

В Hermes инструменты объединены в toolsets. Например, набор `file` включает работу с чтением, записью, поиском и изменением файлов, набор `terminal` отвечает за команды и процессы, а `delegation` предоставляет `delegate_task`. Для задач программной разработки существует составной toolset `coding`, который объединяет несколько возможностей. (github.com)

То есть модель остаётся системой принятия решений, а Tool становится способом воздействовать на внешнюю среду.

7. CLI, API и Git как разные типы инструментов

Когда агент работает с разработкой, его инструменты могут обращаться к совершенно разным интерфейсам.

CLI-инструмент взаимодействует с системой через командную строку. Например, агент может запустить тесты, линтер или команду диагностики. Здесь результатом обычно становится stdout, stderr и код завершения процесса.

API-инструмент работает через программный интерфейс внешней системы. В этом случае модель не обязана знать детали HTTP-запроса. Для неё может существовать функция вроде `get_issue`, а уже обработчик инструмента самостоятельно обращается к API.

Git тоже можно рассматривать как набор операций над системой контроля версий. Для агента особенно важны действия вроде получения diff или статуса репозитория. Причём чтение состояния Git принципиально отличается по риску от `push`, `reset` или переписывания истории.

В Hermes инструменты поэтому нужно оценивать не только по функциональности, но и по возможным последствиям выполнения.

8. Как LLM понимает доступный инструмент

Чтобы модель могла использовать Tool, недостаточно написать Python-функцию и дать ей имя. Модель должна получить описание того, что делает функция и какие аргументы она принимает.

Обычно инструмент представляется модели через schema. В ней есть имя, описание и структура параметров. Например, поисковый инструмент может принимать строку `query`, а инструмент получения diff — путь или область изменений.

Описание здесь имеет очень большое значение. Если Tool называется неясно и его назначение описано слишком общо, модель может вызывать его в неподходящих ситуациях.

Поэтому хорошее описание отвечает на вопрос не только «что технически делает функция», но и «когда модель должна её использовать».

В нашей предыдущей RAG-практике мы как раз описывали `codebase_search` так, чтобы агент понимал: этот инструмент предназначен для получения релевантного контекста кодовой базы перед ответами о структуре и связях проекта. Это и есть пример функции, написанной не только для Python, но и для LLM.

9. Валидация аргументов инструмента

Следующая проблема возникает после того, как модель решила вызвать функцию. Она должна сформировать параметры, соответствующие ожидаемой структуре.

Представим инструмент, принимающий `top_k` от 1 до 8. Модель может случайно передать строку вместо числа или значение 1000. Если обработчик без проверки использует этот параметр, мы получаем непредсказуемое поведение.

Поэтому инструменты должны иметь валидацию аргументов. Schema помогает модели сформировать корректный вызов, но обработчик всё равно должен проверять входные данные.

В Hermes tool runtime оборачивает ошибки выполнения и возвращает их модели в контролируемом формате. Это позволяет агенту увидеть, что вызов оказался неправильным, и при необходимости сформировать новый. (github.com)

Но лучший Tool не полагается только на обработку исключения. Если мы знаем допустимый диапазон или обязательные параметры, эти ограничения должны быть явно выражены в schema и дополнительно проверены внутри обработчика.

10. Skill и Tool — не одно и то же

В завершение важно разделить понятия Skill и Tool.

Tool предоставляет действие. Например, прочитать файл, выполнить команду, получить Git diff или найти строку в кодовой базе.

Skill описывает способ решения повторяющегося типа задач. В Hermes Skills выступают как процедурная память. Skill может содержать инструкции, шаблоны, вспомогательные материалы и сценарий того, как агенту следует работать с определённым классом задач. Hermes предоставляет отдельные инструменты для просмотра и управления Skills. (github.com)

Например, `git diff` сам по себе является действием. Но процедура полноценного code review может быть Skill, который объясняет агенту, когда получать diff, что проверять в изменениях, когда запускать тесты и как оформлять результат.

Получается важное разделение. Tool отвечает на вопрос «что агент технически может сделать», а Skill — «как агенту следует использовать доступные возможности для решения определённой задачи».

В следующей практике мы как раз перейдём от этой теории к собственным инструментам разработки. Мы зарегистрируем операции для получения `git diff`, поиска через `grep` или его безопасный аналог и запуска линтера, после чего проверим, может ли Hermes выбрать подходящий инструмент, правильно сформировать аргументы и использовать полученный результат внутри своего рабочего цикла.
